PYTHON PROGRAMMING CASE STUDIES REPORT

Simple and Medium Level Case Studies

Name	K Ramya sai Nandini
Roll No.	26001082
Branch	BTech
Course	CSE AIML
Section	4
Academic Year	2026

Academic Year 2026

Contents

1. Examination Result Analysis

2. Attendance Calculation

3. ATM Transaction Simulation

4. Employee Payroll System

5. Retail Billing System

6. Student Performance Analysis

7. Food Waste Monitoring System

8. Student Marks Matrix Analysis

9. Tuple-Based Product Catalogue Audit

10. Smart Canteen Order and Stock Management

Introduction

This report presents ten Python case studies designed to practise basic and intermediate programming. The examples demonstrate input and output, arithmetic operations, conditional statements, loops, lists, nested lists, tuples, and simple real-world calculations.

How to Execute the Programs

Use Python 3.x. Save each code sample in a separate .py file, run it using a Python interpreter, and provide the sample input when prompted. No third-party libraries are required.

1. Examination Result Analysis

Problem: Calculate total marks, average marks, and the result based on marks in five subjects. A student passes when the minimum subject mark is at least 35.

Concepts covered: Lists, input(), sum(), min(), arithmetic operations, and conditional statements.

Python Code

```
marks = []

for i in range(5):
    m = int(input("Enter subject marks: "))
    marks.append(m)

total = sum(marks)
average = total / 5

print("Total Marks:", total)
print("Average:", average)

if min(marks) >= 35:
    print("Result: PASS")
else:
    print("Result: FAIL")

if average >= 90:
    print("Grade: A")
elif average >= 75:
    print("Grade: B")
elif average >= 50:
    print("Grade: C")
else:
    print("Grade: D")
```

Sample Output

```
Enter subject marks: 90
Enter subject marks: 85
Enter subject marks: 78
Enter subject marks: 88
Enter subject marks: 95

Total Marks: 436
Average: 87.2
Result: PASS
Grade: B
```

2. Attendance Calculation

Problem: Calculate the attendance percentage and check whether a student is eligible to appear for the examination. The eligibility threshold is 75%.

Concepts covered: User input, division, percentage calculation, and if-else decision making.

Python Code

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

percentage = (attended / total) * 100

print("Attendance:", percentage, "%")

if percentage >= 75:
    print("Eligible for examination")
else:
    print("Not eligible for examination")
```

Sample Output

```
Enter total classes: 100
Enter attended classes: 85

Attendance: 85.0 %
Eligible for examination
```

3. ATM Transaction Simulation

Problem: Simulate a deposit and a withdrawal, then display the final account balance. A withdrawal is processed only when sufficient balance is available.

Concepts covered: Variables, arithmetic assignment operators, input(), and conditional statements.

Python Code

```
balance = 5000

deposit = int(input("Enter deposit amount: "))
balance += deposit

withdraw = int(input("Enter withdrawal amount: "))

if withdraw <= balance:
    balance -= withdraw
    print("Withdrawal successful")
else:
    print("Insufficient balance")

print("Final Balance:", balance)
```

Sample Output

```
Enter deposit amount: 2000
Enter withdrawal amount: 1500

Withdrawal successful
Final Balance: 5500
```

4. Employee Payroll System

Problem: Calculate an employee's salary using basic pay, allowances, a fixed bonus, and deductions. HRA is 20% of basic salary, DA is 10%, and deduction is 5% of gross salary.

Concepts covered: Variables, percentage calculations, arithmetic operations, and formatted output.

Python Code

```
basic = int(input("Enter basic salary: "))
hra = basic * 0.20
da = basic * 0.10
bonus = 2000

gross = basic + hra + da + bonus
deduction = gross * 0.05
net = gross - deduction

print("Basic Salary:", basic)
print("HRA:", hra)
print("DA:", da)
print("Bonus:", bonus)
print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net)
```

Sample Output

```
Enter basic salary: 20000

Basic Salary: 20000
HRA: 4000.0
DA: 2000.0
Bonus: 2000
Gross Salary: 28000.0
Deduction: 1400.0
Net Salary: 26600.0
```

5. Retail Billing System

Problem: Calculate the total shopping bill from multiple products and apply a discount. A 10% discount is applied when the total is at least 2000; otherwise, a 5% discount is applied.

Concepts covered: For loops, accumulation, multiplication, and conditional discount calculation.

Python Code

```
total = 0

n = int(input("Enter number of products: "))

for i in range(n):
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))
    total += price * quantity

if total >= 2000:
    discount = total * 0.10
else:
    discount = total * 0.05

final = total - discount

print("Total Amount:", total)
print("Discount:", discount)
print("Final Bill:", final)
```

Sample Output

```
Enter number of products: 2
Enter product price: 1000
Enter quantity: 1
Enter product price: 600
Enter quantity: 2

Total Amount: 2200.0
Discount: 220.0
Final Bill: 1980.0
```

6. Student Performance Analysis

Problem: Compare a list of student marks, calculate the total and average, and identify the highest and lowest scores. Classify each mark as Excellent, Good, or Needs Improvement.

Concepts covered: Lists, sum(), len(), max(), min(), loops, and if-elif-else statements.

Python Code

```
marks = [85, 90, 75, 95, 80]

total = sum(marks)
average = total / len(marks)
highest = max(marks)
lowest = min(marks)

print("Total Marks:", total)
print("Average Marks:", average)
print("Highest Marks:", highest)
print("Lowest Marks:", lowest)

for m in marks:
    if m >= 85:
        print(m, "Excellent")
    elif m >= 50:
        print(m, "Good")
    else:
        print(m, "Needs Improvement")
```

Sample Output

```
Total Marks: 425
Average Marks: 85.0
Highest Marks: 95
Lowest Marks: 75

85 Excellent
90 Excellent
75 Good
95 Excellent
80 Good
```

7. Food Waste Monitoring System

Problem: Calculate leftover food and the waste percentage for each day. Mark a day as having high food waste when the waste percentage is greater than 20%.

Concepts covered: Parallel lists, indexing, loops, subtraction, percentage calculation, and threshold checking.

Python Code

```
prepared = [100, 120, 150, 100, 200]
served = [90, 100, 120, 95, 150]

for i in range(5):
    waste = prepared[i] - served[i]
    percentage = (waste / prepared[i]) * 100

    print("Day", i + 1)
    print("Food Waste:", waste)
    print("Waste Percentage:", percentage)

    if percentage > 20:
        print("High Food Waste")
    else:
        print("Acceptable Food Waste")
```

Sample Output

```
Day 1
Food Waste: 10
Waste Percentage: 10.0
Acceptable Food Waste

Day 2
Food Waste: 20
Waste Percentage: 16.666666666666664
Acceptable Food Waste

Day 3
Food Waste: 30
Waste Percentage: 20.0
Acceptable Food Waste

Day 4
Food Waste: 5
Waste Percentage: 5.0
Acceptable Food Waste

Day 5
Food Waste: 50
Waste Percentage: 25.0
High Food Waste
```

8. Student Marks Matrix Analysis

Problem: Store the marks of multiple students in a nested list, calculate each student's total and average, calculate subject-wise totals, and identify the student with the highest total.

Concepts covered: Nested lists, nested loops, row-wise and column-wise aggregation, and key-based max().

Python Code

```
marks = [
    [85, 90, 80],
    [75, 88, 92],
    [95, 90, 98]
]

for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])

    print("Student", i + 1)
    print("Total:", total)
    print("Average:", average)

subject_totals = [0, 0, 0]

for row in marks:
    for j in range(3):
        subject_totals[j] += row[j]

print("Subject Totals:", subject_totals)

print("Top Student:",
      marks.index(max(marks, key=sum)) + 1)
```

Sample Output

```
Student 1
Total: 255
Average: 85.0

Student 2
Total: 255
Average: 85.0

Student 3
Total: 283
Average: 94.33333333333333

Subject Totals: [255, 268, 270]
Top Student: 3
```

9. Tuple-Based Product Catalogue Audit

Problem: Store product name, price, and stock quantity as tuples. Calculate each product's inventory value, identify products requiring restocking when stock is below 10, and calculate the total inventory value.

Concepts covered: Tuples, tuple unpacking, iteration, multiplication, accumulation, and stock threshold checking.

Python Code

```
products = [
    ("Pen", 10, 50),
    ("Book", 100, 5),
    ("Bag", 500, 2),
    ("Pencil", 5, 100)
]

total_value = 0

for name, price, stock in products:
    value = price * stock
    total_value += value

    print("Product:", name)
    print("Price:", price)
    print("Stock:", stock)
    print("Value:", value)

    if stock < 10:
        print("Restock Required")
    else:
        print("Stock Available")

print("Total Inventory Value:", total_value)
```

Sample Output

```
Product: Pen
Price: 10
Stock: 50
Value: 500
Stock Available

Product: Book
Price: 100
Stock: 5
Value: 500
Restock Required

Product: Bag
Price: 500
Stock: 2
Value: 1000
Restock Required

Product: Pencil
Price: 5
Stock: 100
Value: 500
Stock Available

Total Inventory Value: 2500
```

10. Smart Canteen Order and Stock Management

Problem: Create a simple canteen ordering system. Accept multiple orders, calculate each item bill, check item availability and stock, update stock after successful orders, and display the total bill and remaining stock.

Concepts covered: Lists, searching with in, index(), loops, stock validation, list updates, and billing.

Python Code

```
menu = ["Idli", "Dosa", "Meals"]
prices = [30, 50, 100]
stock = [20, 15, 10]

total = 0

n = int(input("Enter number of orders: "))

for i in range(n):
    item = input("Enter item name: ")
    quantity = int(input("Enter quantity: "))

    if item in menu:
        index = menu.index(item)

        if quantity <= stock[index]:
            bill = prices[index] * quantity
            total += bill
            stock[index] -= quantity

            print("Order Successful")
            print("Item Bill:", bill)
        else:
            print("Insufficient Stock")
    else:
        print("Item Not Available")

print("Total Bill:", total)
print("Remaining Stock:", stock)
```

Sample Output

```
Enter number of orders: 2

Enter item name: Idli
Enter quantity: 5
Order Successful
Item Bill: 150

Enter item name: Dosa
Enter quantity: 3
Order Successful
Item Bill: 150

Total Bill: 300
Remaining Stock: [15, 12, 10]
```

Conclusion

The ten case studies provide practical experience in solving small programming problems using Python. They show how core language features can be combined to analyse student results, calculate attendance, simulate transactions, process payroll and bills, monitor food waste, analyse tabular marks, audit stock, and manage canteen orders.

Learning Outcomes

By completing these exercises, the student practises writing structured Python programs, processing collections, using loops and conditions, performing calculations, and presenting results clearly.

Submitted by: K Ramya sai Nandini

Roll No.: 26001082

Course: CSE AIML | **Section:** 4 | **Academic Year:** 2026